1

What is the CNF-SAT problem, and what is the #F problem?

CNF-SAT stands for Conjunctive Normal Form Satisfiability problem. The problem is given a set of boolean variables $X = \{x_1, x_2, \dots, x_n\}$ and m clauses where a clause contains literals as a subset of X and the clause contains these literals or its negation and they are all $\vee$'d together. The CNF-SAT problem for a given set of clauses and literals is true iff there exists an assignment true or false for the X such that all clauses are true. The paper writes it as:

$$F(x_1, \dots, x_n) = \wedge_{e=1}^{m}[(x_{ie,1} \oplus b_{e,1}) \vee \cdots \vee (x_{ie,k} \oplus b_{e,k})]$$

Where $b_{e,k} \in \{0, 1\}$ acting as negating or not negating the specific variable $x_{ie,j}$ with $ie, j \in \{1, \dots, n\}$. This is a special case of the CNF-SAT problem by constricting all clauses to be of size k called k-CNF-SAT which can be used to solve CNF-SAT (proved in CSDS343) and vice versa.

Now the #F problem looks at the number of assignments to the variables X that result in all clauses being true or the function above as true. Note that the total number of assignments is 2^n but if #F> 0 then we are able to return true for that specific problem on those clauses and variables.

2

What is the main claim of the paper (i.e. expand what it means for the ETH to be false)?

The main claim of the paper is that it does not take $2^{\Omega(n)}$ time in the worst case to decide the $3-SAT$ problem. In other words, as we increase the number of variables the time it takes to solve our problem is multiplied by 2. I believe there are some algorithms that make the multiplier less than 2 but it is still exponential time since the multiplier is greater than 1 and scales with n. ETH being false means that there exists an algorithm that does not grow at this exponential rate. The actual algorithm the paper proposes is in $O((n+m) \times 2^{n/(\log n)^{0}.5})$ with some extra terms that dont matter as much as the key point that this runtime is "sub-exponential". While this is not polynomial runtime so $P \neq NP$ is still an open question, we now have an algorithm that runs quicker than exponential time (with a few large constant terms). I believe it was in Anthropic's paper on the same topic (supposedly Anthropic and OpenAI got the same conclusion with thier AI models within days of eachother) noted that their algorithm is only relevant for $n > 10^{1000}$ and otherwise simple brute force is faster. The bigger implications of this is that there are a wide

variety of papers are dependent on the ETH being true as a computational lower bound, like the strong ETH, GAP-ETH, NSETH and others that the OpenAI paper mention. In other words, many theorems are now false because they either rely on ETH being true OR are a stronger conjecture.

3

What are the classes P, NP, and #P?

P refers to problems that can be solved/decided in polynomial time. IE given a problem, find a valid solution or determine no such solution exists. An example problem are a lot of the algorithms we make in class / HW like finding a polynomial time algorithm for graph isomorphism given a tree width k for one of the input graphs G, the valid solution in this case would be the actual assignment of nodes in G to nodes in H such that the graphs are the same.

NP refers to problems that can be verified in polynomial time. IE given a problem and a proposed solution, determine that the solution is a valid solution to the problem. An example problem would be the SAT problem because if we are given an assignment of the variables X we could determine in polynomial time if the given solution does indeed have each clause be true.

#P refers to the number of solutions to a given input problem. IE given a problem, how many solutions does it have. Example being #F from above look at how many distinct assignments to the vairables do we have that return true.

4

What do you know about polynomial time reductions of NP-complete problems from what you may have studied in a previous class, and from this knowledge why does the ETH being false not imply that all problems in NP can be solved in sub-exponential time?

From CSDS343, a problem A is NP-complete if there exists a polynomial time reduction from an NP-complete problem or all NP problems to our problem A and A is in NP so it can be verified in polynomial time.

Even though we have a sub-exponential time algorithm for an NP-complete problem (3-SAT or k-SAT), we do not have a strong way of enforcing this sub-exponential time for all reductions. What I mean by this is suppose the reduction from problem B to 3-SAT results in a 3-SAT of size n^{100}. This is still a polynomial time reduction but now our current runtime for our problem B is $O(n^{100}2^{n^{100}/(logn)^{0}.5)}$ which is

definitely exponential time and arguably much worse than exponential time in this case because 100 was arbitrary and could be much much larger.

The reason why this is not a problem typically when looking at NP-completeness is because n^k is polynomial and so is n^{r^k} where r is the reduction polynomial. So even though our polynomial gets really really large, it is still a polynomial so it is considered "fine". However in the context of sub-exponential problems where the polynomial reduction does not neccessarily hold the sub-exponential time property this does cause a problem and is why ETH being false does not imply all problems in NP can be solved in sub-exponential time.